

SIMATS ENGINEERING

ASSESSMENT TOOL 3 – CASE STUDY

COURSE CODE: ITA14 **COURSE NAME:** Ethical Hacking

COURSE OUTCOME COVERED

CO4: Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux. (BT4, BT5)

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Total Marks:**100

Code of Conduct:

I _____ K. HARSHIKA (192473020) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ____HARSHIKAREDDY_____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity, difficult to understand

1. A mid-sized e-commerce company hosts its web application on an Apache server running on a Linux environment. Recently, customers reported unauthorized transactions and abnormal account activities. Upon investigation, the IT team discovered suspicious HTTP requests in server logs, including attempts to access hidden directories and injection of malicious scripts into input fields. Additionally, the server is running an outdated version with known vulnerabilities, and no Web Application Firewall (WAF) is implemented. As a cybersecurity analyst, you are tasked with conducting a comprehensive security assessment using Kali Linux. Analyze the possible attack vectors exploited by the attacker, including web server vulnerabilities, input validation issues, and outdated software components. Explain in detail how you would perform reconnaissance, vulnerability scanning, and exploitation using appropriate Kali Linux tools. Further, propose a layered defense strategy that includes server hardening, secure coding practices, and network-level protections to prevent such incidents in the future.

1.1 Possible Attack Vectors

- Reconnaissance-enabled directory disclosure – hidden/backup directories (e.g., /admin, /backup, /.git) were discoverable, giving the attacker a map of the application.
- Unpatched Apache and framework components – the outdated software version indicates known CVEs (e.g., outdated Apache/PHP modules) that allow remote code execution or information disclosure.
- Improper input validation – unsanitised input fields allowed injection of malicious scripts (Cross-Site Scripting) and potentially command/SQL injection.
- Absence of a Web Application Firewall (WAF) – no layer-7 filtering existed to detect or block malicious payloads before they reached the application.
- Weak session/authentication controls – abnormal account activity and unauthorized transactions suggest session hijacking or credential-stuffing following the above weaknesses.

1.2 Security Assessment Methodology on Kali Linux

Phase 1 – Reconnaissance (Information Gathering)

Goal: build a profile of the target without/with minimal interaction.

```
whois example.com
nmap -sS -sV -O -p- example.com          # port/service/OS fingerprint
whatweb example.com                     # web technology fingerprint
gobuster dir -u https://example.com -w /usr/share/wordlists/dirb/common.txt -x php,bak
```

This reveals open ports, the Apache version banner, server-side language, and hidden directories such as /admin or /backup that the attacker had abused.

Phase 2 – Vulnerability Scanning

```
nikto -h https://example.com -Tuning 123
nmap --script vuln example.com
```

Nikto flags outdated server banners, default files, missing security headers, and known CVEs. An authenticated scan with OpenVAS/Greenbone can be layered on for a CVSS-scored vulnerability list, which is used to prioritise remediation.

Phase 3 – Exploitation / Validation (authorised test environment only)

- Confirm reflected/stored XSS by submitting a harmless marker string into each input field and observing whether it is rendered unescaped in the response.
- Verify directory-listing/backup-file exposure identified by gobuster/nikto by requesting the discovered paths directly and confirming sensitive content is returned.
- Match the fingerprinted Apache/module version against the National Vulnerability Database / Exploit-DB to confirm whether a known, patchable CVE applies — used only to prove impact for the report, not to cause damage.
- Use Metasploit's auxiliary scanner modules (e.g., auxiliary/scanner/http/apache_userdir_enum) in the lab copy of the environment to validate findings under a signed rules-of-engagement document.

1.3 Layered (Defense-in-Depth) Remediation Strategy

Server Hardening

- Patch Apache and all modules/CMS plugins to the latest stable release; enable automatic security updates.
- Disable directory listing (Options -Indexes) and remove/relocate backup and admin paths outside the web root.
- Run the service as a low-privilege user, enable SELinux/AppArmor, and remove unused modules.
- Enforce HTTPS with HSTS and disable weak TLS ciphers.

Secure Coding Practices

- Validate and sanitise all input server-side (allow-list validation), and encode all output to prevent stored/reflected XSS.
- Use parameterised queries/ORM to remove injection risk, and return generic error messages (no stack traces or DB details).
- Adopt secure session management: HttpOnly + Secure cookies, session regeneration after login, short idle timeouts.

Network-Level Protections

- Deploy a Web Application Firewall (e.g., ModSecurity with the OWASP Core Rule Set) in front of the application.
- Add network segmentation, a reverse proxy/CDN, and rate-limiting to blunt automated attacks.
- Enable centralised logging and a SIEM/IDS (e.g., Wazuh, Suricata) for continuous monitoring and alerting on suspicious HTTP patterns.
- Schedule recurring vulnerability scans and an annual/bi-annual third-party penetration test.

2. A financial services web application allows users to log in, view account details, and perform transactions. A recent penetration test revealed that the application is vulnerable to SQL injection and Cross-Site Scripting (XSS) attacks. Error messages expose database structure information, and user inputs are not properly sanitized. Attackers have exploited these weaknesses to extract sensitive customer data and inject malicious scripts into web pages viewed by other users. You are assigned to perform a detailed security audit of the application. Question: Examine how SQL injection and XSS vulnerabilities can be exploited in this scenario. Describe a step-by-step methodology using Kali Linux tools such as Burp Suite and SQLmap to identify and validate these vulnerabilities. Evaluate the impact of such attacks on confidentiality, integrity, and availability (CIA triad). Recommend comprehensive mitigation strategies, including input validation, output encoding, secure session handling, and implementation of security controls.

2.1 How the Vulnerabilities Can Be Exploited

SQL Injection: Because user input (e.g., login fields, account-search parameters) is concatenated directly into SQL queries, an attacker can inject SQL meta-characters to alter query logic — for example bypassing authentication with a always-true condition, or using UNION-based injection to pull data from other tables once the number of columns is known. Verbose error messages that reveal table/column names make this reconnaissance trivial for the attacker.

Cross-Site Scripting (XSS): Because output is not encoded, a script payload submitted through a field such as a transaction note or profile field is stored and later rendered inside another user's (or administrator's) browser session, allowing session-cookie theft, forged transactions, or credential harvesting via a fake login form.

2.2 Step-by-Step Assessment Methodology (Kali Linux)

Step 1 – Proxy-based Mapping with Burp Suite

- Configure the browser to route traffic through Burp Suite's intercepting proxy (127.0.0.1:8080).
- Use Burp Spider/Crawler to map every endpoint, form, and parameter of the application.
- Send interesting requests to Burp Repeater to manually manipulate parameters (e.g., appending a single quote ' to a field and observing whether a database error is returned).
- Run Burp Scanner (or the free Intruder module with payload lists) to fuzz each parameter with SQLi/XSS test strings and flag anomalous responses (error text, time delays, reflected payloads).

Step 2 – Confirming SQL Injection with SQLmap

Once Burp identifies a parameter that behaves unusually, the request is exported/copied and validated with SQLmap:

```
sqlmap -u "https://bank.example.com/account?id=1" --batch --level=3 --risk=2
sqlmap -u "https://bank.example.com/account?id=1" --dbs
sqlmap -u "https://bank.example.com/account?id=1" -D bankdb --tables
sqlmap -u "https://bank.example.com/account?id=1" -D bankdb -T customers --columns
```

In an authorised engagement, --dump can be used against a single non-production table only to demonstrate impact for the report; the objective is proof of exploitability, not data exfiltration.

Step 3 – Confirming XSS

- Submit a benign detection payload (e.g., a unique marker string wrapped in a script tag) into each input field and check the rendered HTML source for un-encoded reflection.
- Use Burp's built-in XSS detection and browser DevTools to confirm whether the payload executes in the DOM, and whether it is stored (persists for other users) or reflected (only in the immediate response).

2.3 Impact on the CIA Triad

- Confidentiality: SQL injection allows extraction of customer PII and financial data; XSS-based session hijacking exposes authenticated account data.
- Integrity: Injection can modify or delete records (e.g., altering balances or transaction logs); stored XSS can silently alter what other users see or submit.
- Availability: Destructive injection statements or resource-heavy queries can degrade or crash the database/application, denying service to legitimate customers.

2.4 Mitigation Strategy

- Input validation: strict allow-list/type validation on every parameter; reject unexpected characters server-side.
- Parameterised queries / prepared statements or an ORM to eliminate SQL injection at the source; least-privilege database accounts.
- Output encoding (context-aware HTML/JS/URL encoding) plus a strict Content-Security-Policy to neutralise XSS payloads.
- Secure session handling: HttpOnly, Secure, SameSite cookies; session regeneration on privilege change; short timeouts and re-authentication for high-value transactions.
- Generic error handling that never surfaces database schema or stack traces to the client.
- Defense-in-depth controls: a WAF, regular SAST/DAST scanning in the CI/CD pipeline, and periodic penetration testing.

3. A corporate office provides wireless internet access to employees using WPA2-PSK authentication. Recently, several employees reported unusual login prompts and slow network performance. A security investigation revealed the presence of a rogue access point (Evil Twin) mimicking the official network SSID. Attackers used this setup to capture login credentials and perform man-in-the-middle (MITM) attacks. You are tasked with analyzing the breach and strengthening wireless security. Explain how the attacker successfully executed the Evil Twin attack and intercepted sensitive data. Describe how Kali Linux tools such as Aircrack-ng and Wireshark can be used to detect and analyze such attacks. Evaluate the weaknesses in the existing wireless security configuration. Propose robust countermeasures, including secure authentication mechanisms, network monitoring, and user awareness strategies.

3.1 How the Attack Was Executed

802.11 management frames (beacons, probe responses, deauthentication) are unauthenticated and unencrypted. This allowed the attacker to clone the legitimate SSID and BSSID characteristics onto a rogue access point with a stronger signal, then send spoofed deauthentication frames to force connected clients off the genuine AP. Client devices, which trust SSIDs by name rather than by cryptographic identity, automatically reconnect to whichever AP responds first — the rogue one. Because the fake AP does not need to know the real WPA2-PSK passphrase (it can present an open network or a captive portal), victims are prompted for credentials that are then captured, or their traffic is simply relayed (man-in-the-middle) to the real network while being sniffed in transit.

3.2 Detection and Analysis Using Kali Linux

Aircrack-ng suite

```
airmon-ng start wlan0          # enable monitor mode
airodump-ng wlan0mon          # list nearby BSSIDs/SSIDs and clients
```

Reviewing the airodump-ng output for duplicate SSIDs with different BSSIDs, mismatched channel/vendor OUI, or an unusually strong-signal AP is the primary way to spot an Evil Twin. aireplay-ng can be used in a controlled lab to reproduce the deauthentication behaviour for training/analysis purposes so responders recognise the pattern in live captures.

Wireshark

- Capture on the monitor-mode interface and filter on wlan.fc.type_subtype == 0x0c (deauthentication frames) to identify an abnormal burst of deauth traffic.
- Inspect beacon frames for two access points broadcasting the identical SSID with different BSSID/MAC or capability flags — a strong Evil Twin indicator.
- Look for clients associating with an unfamiliar BSSID immediately after a deauth burst — evidence of forced roaming to the rogue AP.

3.3 Weaknesses in the Existing Configuration

- WPA2-PSK uses a single shared passphrase with no per-user identity, so a rogue AP can impersonate the network without needing to break the encryption at all.
- Unauthenticated management frames (no 802.11w) allow anyone to forge deauthentication frames.
- No wireless intrusion detection/prevention system (WIDS/WIPS) was in place to alert on rogue APs or deauth floods.
- No employee awareness training on verifying network authenticity or recognising unexpected login prompts.

3.4 Recommended Countermeasures

- Migrate to WPA2/WPA3-Enterprise with 802.1X and certificate-based authentication (EAP-TLS), removing the shared-secret weakness entirely.
- Enable Protected Management Frames (802.11w) to make deauthentication frames unforgeable.
- Deploy a WIDS/WIPS to continuously scan for rogue BSSIDs and automatically alert/contain them.
- Enforce certificate pinning/VPN for sensitive applications so credentials are never usable even if captured on a rogue portal.
- Run periodic wireless security audits and user-awareness training on identifying suspicious captive portals or repeated re-authentication prompts.

4. An organization's internal network hosts a web-based employee management system accessible over Wi-Fi. The wireless network uses weak passwords, and the web application lacks proper authentication and session management. An attacker gains access to the Wi-Fi network, performs reconnaissance, and exploits vulnerabilities in the web application to escalate privileges and access confidential employee data. You are assigned to perform a full penetration testing assessment. Analyze the attack chain starting from wireless network compromise to web application exploitation. Describe each phase of the attack, including reconnaissance, scanning, exploitation, and privilege escalation. Demonstrate how Kali Linux tools can be used in each phase. Propose a comprehensive security framework that addresses both wireless and web application vulnerabilities, ensuring defense-in-depth.

4.1 Attack Chain Overview

The attack progresses through four linked phases: wireless reconnaissance and credential compromise, network-level scanning, web application exploitation, and privilege escalation to confidential data. Each phase amplifies the access gained in the previous one, which is the core rationale for a defense-in-depth response.

Phase 1 – Wireless Reconnaissance and Access

```
airmon-ng start wlan0
```

```
airodump-ng wlan0mon # identify target SSID/BSSID and clients
airodump-ng -c <ch> --bssid <BSSID> -w capture wlan0mon # capture WPA handshake
aircrack-ng capture-01.cap -w rockyou.txt # dictionary attack against the weak PSK
```

A weak, dictionary-guessable Wi-Fi password allows the handshake captured above to be cracked offline, giving the attacker network-layer access without ever touching the office physically.

Phase 2 – Internal Network Scanning

```
nmap -sn 192.168.1.0/24 # host discovery
nmap -sV -sC -p- 192.168.1.10 # service/version enumeration of the web server
```

This reveals the employee management system's IP, open ports, and the web stack in use.

Phase 3 – Web Application Exploitation

- Map the login and session workflow with Burp Suite; because the application lacks proper authentication and session controls, test for missing account-lockout, predictable session tokens, and direct object references (e.g., changing a numeric employee ID in the URL).
- Attempt credential attacks (e.g., Hydra against the login form) if weak/default credentials are suspected, and validate injection points with SQLmap where applicable.
- Exploit an Insecure Direct Object Reference (IDOR) or missing role check to view other employees' records, or hijack a predictable session ID to impersonate a logged-in HR/admin user.

Phase 4 – Privilege Escalation

- If a code-execution flaw is confirmed in the lab copy, a foothold shell is validated (e.g., via a Metasploit module matched to the identified CVE) strictly within the authorised scope.
- Local privilege-escalation checks are run with enumeration scripts such as LinPEAS/WinPEAS to identify misconfigurations (weak file permissions, exposed credentials, unpatched kernel) that would let a low-privilege web-app account become an administrator.
- Escalated access is used only to demonstrate the ability to reach the confidential employee database, then the finding is documented — not exploited further.

4.2 Defense-in-Depth Security Framework

Wireless Layer

- Strong, high-entropy PSKs or WPA2/WPA3-Enterprise with 802.1X; disable WPS; segregate guest/employee SSIDs.
- WIDS/WIPS monitoring and regular Wi-Fi password rotation.

Network Layer

- VLAN segmentation isolating the employee management system from general Wi-Fi traffic; internal firewalling and NAC (network access control).
- Host and service hardening, disabling unused ports/services surfaced by nmap.

Application Layer

- Strong authentication (MFA), robust session management (secure, expiring, unpredictable tokens), and enforced role-based access control (RBAC) to close the IDOR gap.
- Input validation, parameterised queries, and regular SAST/DAST + code review.

Host / Privilege Layer

- Least-privilege service accounts, timely OS/application patching, and file-permission audits to close common privilege-escalation paths.
- Centralised logging/SIEM correlating wireless, network, and application events so a multi-stage attack like this one is detected early, not only after data loss.

5. A university campus experiences frequent Wi-Fi disruptions during peak hours. Investigation reveals repeated deauthentication attacks targeting access points, causing legitimate users to disconnect. The attacker uses automated tools to continuously send deauth packets, leading to denial of service. You are tasked with identifying the cause and implementing solutions. Explain how deauthentication attacks are carried out in wireless networks and how attackers exploit protocol weaknesses. Describe how Kali Linux tools can be used to simulate and analyze such attacks in a controlled environment. Evaluate the impact on network availability and user productivity. Recommend effective mitigation strategies, including network configuration changes, intrusion detection systems, and secure wireless protocols.

5.1 How Deauthentication Attacks Work

The IEEE 802.11 standard's management frames — including the deauthentication and disassociation frames — are transmitted in the clear and are not authenticated in legacy WPA2 configurations. Any device within radio range can spoof the source (client or AP) MAC address and broadcast deauthentication frames on behalf of the access point. Because client devices are designed to immediately honour a deauthentication frame (assuming it genuinely came from the AP), an attacker can continuously force every associated client to disconnect and re-authenticate, or refuse to let them stay connected at all — producing a sustained denial of service. This is precisely the protocol weakness — unauthenticated management frames — that automated tools abuse to flood the campus access points.

5.2 Simulating and Analysing the Attack in a Controlled Lab

The following is performed only in an isolated test lab on equipment the tester owns/is authorised to test, never against production access points:

```
airmon-ng start wlan0          # monitor mode
airodump-ng wlan0mon          # identify lab AP's BSSID and channel
aireplay-ng --deauth 10 -a <lab-AP-BSSID> wlan0mon # send a controlled burst of deauth frames
```

Running this against the lab AP while a Wireshark capture is active demonstrates exactly how legitimate clients drop off; the resulting capture (filtered on wlan.fc.type_subtype == 0x0c) becomes the reference pattern used to recognise the same signature in the real campus logs — a sudden, high-rate burst of deauth frames from a single source MAC, unrelated to any genuine AP-initiated event.

5.3 Impact on Availability and Productivity

- Legitimate users are repeatedly dropped and must re-authenticate, collapsing effective throughput during peak hours.
- Time-sensitive academic activity (online exams, class portals, video lectures) is disrupted, and helpdesk load increases from user complaints.

- A sustained deauth flood can also be used as a precursor to an Evil Twin attack (forcing clients toward a rogue AP), compounding the risk beyond simple denial of service.

5.4 Mitigation Strategies

- Enable 802.11w Protected Management Frames (PMF) on all access points so deauthentication/disassociation frames must be cryptographically signed and cannot be forged.
- Migrate to WPA3, which mandates PMF and removes this entire class of attack.
- Deploy a Wireless Intrusion Detection/Prevention System (WIDS/WIPS) to detect abnormal deauth-frame rates and automatically alert network administrators or contain the offending device.
- Increase AP density/channel planning so isolated interference is less disruptive, and enable band-steering to spread load.
- Maintain RF spectrum monitoring and physical-security awareness (rogue devices are usually within physical range), and log/alert on repeated deauth bursts for incident response.

